

UNIVERSIDAD DEL VALLE DE GUATEMALA

Colegio Universitario



Informe del Proyecto 01

(Grupo 02)

Integrantes del grupo:

Diego Fernando Reyes Salazar - 24179

Carlos Emilio Cifuentes Espinoza - 24196

Yu-Fong Azucena Chen Shutuc - 242115

Catedrático: Mario Francisco Barrientos Alejos

Auxiliar: Carlos Fernando Angel Magaña

Facultad de Ingeniería, Base de Datos 1, Sección 10

Contenido

Título	Página
I. Github	3
II. Resumen	3
III. Pregunta inicial	4
IV. Etapa 1: Base de datos	5
a. Fuentes	5
b. Modelo de datos	5 - 6
c. Archivos excluidos	7
d. Problemas y limitaciones	7 - 9
e. Construcción y verificación de la base	10
V. Etapa 2: Queries	11 - 19
VI. Etapa 3: Preguntas propias	20 - 31
VII. Recomendación y conclusión	32 - 34
VIII. Reproducibilidad	35

I. Github

https://github.com/Rey24179/Proyecto_1_Base_Datos_NBA.git

II. Resumen

Se ha documentado el proceso llevado a cabo para la construcción de una base de datos (BDD) PostgreSQL a partir de los archivos CSV de la NBA proporcionados por el curso. Además, se complementó la investigación con herramientas de API para desarrollar un análisis que informa la decisión del equipo sobre en qué equipo se invertiría en la temporada 2021/2022.

Hay tres etapas: la creación de la base de datos, la ejecución de queries para familiarizarse con el modelo de datos propuesto y la justificación de la decisión final, informada por preguntas de nuestra propia creación.

La BDD contiene 62,379 partidos, 9,035 jugadores y 65,158 asignaciones de árbitros, distribuidos en once tablas normalizadas. Se ejecutaron 21 consultas: 8 que respondían a preguntas en la Etapa 2 y 13 propios. Hubo algunos problemas en la calidad de los datos, pero se tomaron los pasos correspondientes para navegar la problemática.

Ya en la Etapa 3, decidimos que es recomendable invertir en Utah Jazz ya que, aunque no es el equipo con más victorias o de mayor crecimiento, es el único que cumple tres características indispensables: rendimiento sostenido, eficiencia en gastos y trayectoria positiva.

III. Pregunta inicial

¿En qué equipo de la NBA invertiríamos para la temporada 2021/2022?

Ya que hay muchos datos en los cuales basar la respuesta a la pregunta planteada, se exploraron ámbitos variados para llegar a una decisión. Uno de los puntos que se tuvieron en cuenta fue la sostenibilidad del equipo. Por ejemplo, que un equipo gane mucho dinero no significa que no tengan gastos que disminuyan en la ganancia. Igualmente, no se quisiera invertir en el equipo más barato porque puede que simplemente sean malos y no rindan al beneficio de los inversores. La consistencia en la mejora también es importante. Si hay una tendencia constante de mejora y esta mejora se mantiene, en vez de fluctuar o irse en declive, es un buen índice también.

Entonces se definieron tres condiciones para informar la decisión:

1. El rendimiento sostenido definido que el equipo gane de forma consistente.
2. El valor por el dinero, lo cual calcula que lo que se les paga conserve cierta ganancia.
3. Y el crecimiento, que la trayectoria del equipo suba y cuente con talento joven que demuestra mejora consistente.

Con las condiciones en mente, las consultas convergen en un índice compuesto y produce la recomendación final.

IV. Etapa 1: Base de Datos

a. Fuentes

Se usaron los archivos CSV "NBA Database" de Wyatt Walsh y la API oficial de la NBA mediante la librería `nba_api`, endpoint `LeagueDashPlayerStats`.

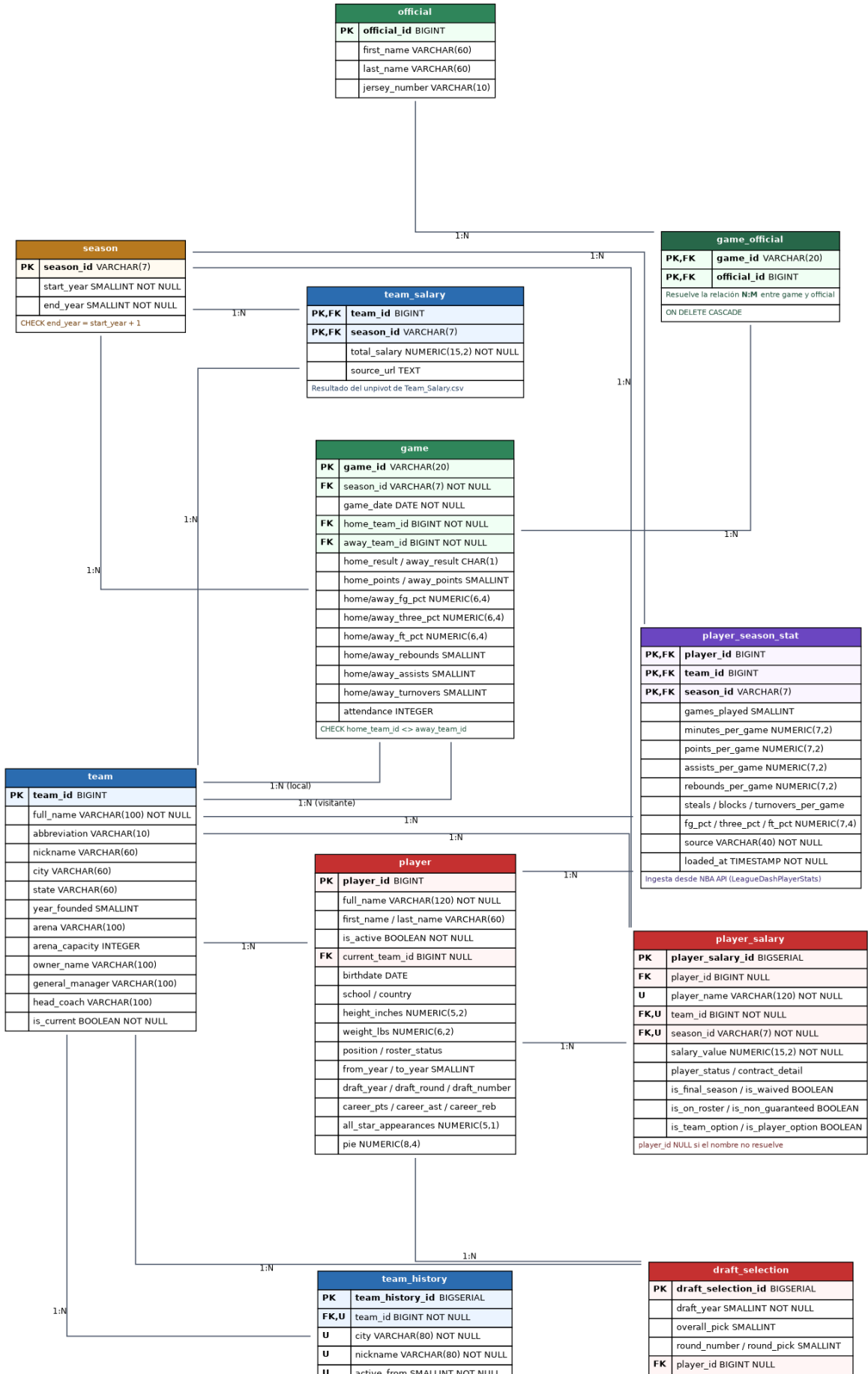
El análisis cubre las temporadas 2015-16 a 2020-21, pero también se conservó el historial de partidos desde 1946 porque descartarlo habría implicado eliminar franquicias históricas necesarias para satisfacer las llaves foráneas.

b. Modelo de datos

El modelo diseñado está conformado por once tablas. Se incorporó la entidad "season" porque este permite relacionar partidos y salarios, aunque provengan de archivos con formatos distintos. Se resuelve la relación muchos a muchos entre partidos y árbitros mediante la tabla "game_official".

Modelo Entidad-Relación — Base de Datos NBA

CC3088 Base de Datos 1 · Proyecto 1 · PostgreSQL · 11 tablas
PK = llave primaria · FK = llave foránea · U = restricción UNIQUE · Cardinalidades 1:1, 1:N y N:M



c. Archivos excluidos

Se excluyó "News.csv" mayormente por su tamaño ya que las laptops que se usaron encontraron dificultades con almacenamiento y procesamiento por ello. Tenía 772 MB y esto conformaba el 93% del peso del conjunto de datos. Además, no aportaba información significativa relacionada a las métricas seleccionadas.

También se excluyeron "Game_Inactive_Players.csv" y "Draft_Combine.csv" ya que no se vió necesario incluir la información de jugadores inactivos.

d. Problemas y limitaciones

1. TEAM_ID = 0 como marcador de "sin equipo"

"Player_Attributes.csv" no usa valores nulos para los jugadores sin equipo, usa el identificador 0. Afecta a 664 jugadores retirados o agentes libres. Como ningún equipo real tiene ese identificador, insertarlo viola la llave foránea hacia team y aborta la carga completa. Se decidió convertir en NULL los 0, conservando a los jugadores necesarios para preguntas históricas.

2. Phoenix Suns tiene dos abreviaturas distintas

"Team_Salary.csv" abrevia a los Suns como PHO mientras que Team.csv usa PHX. El cruce por abreviatura no encontraba correspondencia y descartaba silenciosamente las seis filas de nómina de Phoenix. Se resolvió primero por nombre completo normalizado, así sí coincide en ambos archivos.

3. Colisión de abreviatura entre franquicias

Los Washington Wizards (activos) y los Washington Capitols (innactivos desde 1951) comparten la abreviatura WAS. El diccionario de búsqueda resolvía hacia los Capitols, de modo que los 131 millones de nómina de los Wizards en 2020-21 quedaban registrados en una franquicia extinta,

que además no tiene estado asignado y por lo tanto distorsionaba el cálculo de salarios por estado. Los diccionarios de búsqueda se construyen priorizando equipos vigentes, de modo que una abreviatura ambigua nunca resuelva hacia una franquicia desaparecida.

4. "Game.csv" con partido duplicados

"Game.csv" contiene 69 "GAME_ID" repetidos (138 filas). Las filas duplicadas no son idénticas: difieren en algunas columnas de estadísticas, lo que hacía que temporadas como 2016-17 mostraran 1,231 partidos en lugar de 1,230. Se conservó la fila con mayor cantidad de valores no nulos, por ser la más completa. Resultó en 62,379 partidos únicos.

5. Errores en el campo state

Atlanta Hawks tiene state = 'Atlanta', que es la ciudad y no el estado; el valor correcto es Georgia. Toronto Raptors tiene state = 'Ontario', que es una provincia canadiense y no un estado estadounidense. Se corrigieron Atlanta a Georgia y Toronto como Ontario (Canadá) en las agregaciones por estado.

6. Cobertura parcial de árbitros

"Game_Officials.csv" solo cubre partidos a partir de 1996: 21,711 de los 62,379 registrados. Un JOIN sin advertencia descarta silenciosamente dos tercios del historial. Para las seis temporadas relevantes la cobertura es completa, por lo que no afecta el análisis, pero se documenta la limitación.

7. La columna "all_star_appearances" es inservible

De los 523 jugadores con salario registrado en 2020-21, únicamente 3 tienen un valor mayor que cero en esta columna. Decisión: utilizar PIE (Player Impact Estimate), disponible para 401 de esos 523 jugadores, como medida de valor del jugador en todo el análisis.

8. "Player_Salary.csv" sin identificador de jugador

El archivo solo trae el nombre en texto. El cruce directo resuelve apenas el 74% de los registros, porque los nombres vienen con acentos y guiones eliminados, con sufijos y con marcas pegadas al nombre. Se normalizaron los nombres eliminando acentos y caracteres no alfanuméricos antes de cruzar, lo que eleva la resolución al 93.2% (1,204 de 1,292). De los 88 restantes, 8 son ambiguos y 80 corresponden a novatos de 2020 que no existen en el catálogo. En esos casos se conserva el nombre y el identificador queda nulo, en lugar de asignar una correspondencia incorrecta.

9. Alcance del conjunto de datos: solo temporada regular

Todos los "SEASON_ID" de "Game.csv" inician con el dígito 2, que en la codificación de la NBA corresponde a temporada regular. No hay un solo partido de playoffs ni de pretemporada.

e. Construcción y verificación de la base

Se desarrolló un script en Python que lee directamente el archivo ZIP, aplica las transformaciones necesarias para aclarar los problemas que surgían e inserta los datos en PostgreSQL. El script incluye un modo de validación que reproduce en memoria las restricciones del esquema y aborta sin insertar nada si detecta violaciones.

Conteos obtenidos

Tabla	Filas
draft_selection	7,890
game	62,379
game_official	65,158
official	143
player	9,035
player_salary	1,292
player_season_stat	540
season	80
team	45
team_history	60
team_salary	180

V. Etapa 2: Queries

Usando los queries para familiarizarse con la base de datos, se hicieron las ocho consultas obligatorias. Las consultas en sí se pueden encontrar en database/analysis_queries.sql. Para cada pregunta, se muestra la pregunta, el resultado obtenido y la interpretación.

Las preguntas a responder:

1. ¿Quién es el jugador activo más alto? ¿Y el más bajo?

Categoría	Jugador	Estatura (pulgadas)
Más alto	Tacko Fall	89.00
Más bajo	Isaiah Thomas	69.00
Más bajo	Chris Clemons	69.00

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane displays the database structure, with 'nba_project' selected under 'PostgreSQL 18'. The main pane shows a SQL query in the 'Query' editor. The query is as follows:

```
-- 1. Jugador activo mas alto y mas bajo.
WITH activos AS (
  SELECT full_name, height_inches
  FROM player
  WHERE is_active AND height_inches IS NOT NULL
)
SELECT 'Mas alto' AS categoria, full_name, height_inches
FROM activos
WHERE height_inches = (SELECT MAX(height_inches) FROM activos)
UNION ALL
SELECT 'Mas bajo', full_name, height_inches
FROM activos
WHERE height_inches = (SELECT MIN(height_inches) FROM activos)
ORDER BY categoria DESC, full_name;
```

Below the query editor, the 'Data Output' pane shows the results of the query. It displays three rows, ordered by category (Mas bajo, Mas bajo, Mas alto) and then by full name. The columns are 'categoria', 'full_name', and 'height_inches'.

categoria	full_name	height_inches
Mas bajo	Chris Clemons	69.00
Mas bajo	Isaiah Thomas	69.00
Mas alto	Tacko Fall	89.00

At the bottom of the interface, a status bar indicates 'Successfully run. Total query runtime: 93 msec. 3 rows affected.' and 'Query complete 00:00:00.093'.

2. ¿Cuál fue el promedio de puntos anotados y recibidos por cada equipo en cada una de las temporadas relevantes?

La consulta devuelve 180 filas: 30 equipos por 6 temporadas. Los cinco mejores diferenciales de la temporada 2020-21 son:

Equipo	Anotados	Recibidos	Diferencial
UTA	112.56	103.64	+8.92
MIL	120.13	114.24	+5.89
PHX	113.00	107.17	+5.83
LAC	110.85	105.15	+5.69
PHI	113.64	108.06	+5.58

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the 'Object Explorer' with a tree view of the database structure, including 'Servers (1)', 'PostgreSQL 18', 'Databases (3)', and 'nba_project'. The main window shows a SQL query in the 'Query' tab, which is a PostgreSQL query to calculate the average points scored and received by each team in each season, along with the differential. The query is as follows:

```
-- 2. Promedio de puntos anotados y recibidos por equipo y temporada.
WITH resultados AS (
  SELECT season_id, home_team_id AS team_id, home_points AS favor, away_points AS contra
  FROM game
  UNION ALL
  SELECT season_id, away_team_id, away_points, home_points
  FROM game
)
SELECT r.season_id,
       t.full_name,
       ROUND(AVG(r.favor), 2) AS puntos_anotados,
       ROUND(AVG(r.contra), 2) AS puntos_recibidos,
       ROUND(AVG(r.favor) - AVG(r.contra), 2) AS diferencial
FROM resultados r
JOIN team t ON t.team_id = r.team_id
WHERE LEFT(r.season_id, 4)::int BETWEEN 2015 AND 2020
GROUP BY r.season_id, t.team_id, t.full_name
ORDER BY r.season_id, puntos_anotados DESC;
```

The 'Data Output' tab shows the results of the query, displaying 180 rows. The first five rows are shown in the table below:

season_id	full_name	puntos_anotados	puntos_recibidos	diferencial
2015-16	GSW	114.89	104.13	10.76
2015-16	OKC	110.22	102.94	7.28
2015-16	SAC	106.59	109.06	-2.48
2015-16	HOU	106.55	106.35	0.20
2015-16	BOS	105.72	102.51	3.21

The status bar at the bottom indicates 'Total rows: 180', 'Query complete 00:00:01.415', and 'Successfully run. Total query runtime: 1 secs 415 msec. 180 rows affected.'

3. Muestre el top 5 de árbitros en cuyos juegos pitados el equipo visitante pierde.

Árbitro	Derrotas visitante	Partidos	Porcentaje
Tom Washington	693	1,135	61.06%
James Capers	685	1,082	63.31%
Scott Foster	677	1,147	59.02%
Ed Malloy	674	1,117	60.34%
Pat Fraher	663	1,076	61.62%

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'nba_project' database. The main window shows a SQL query in the 'Query' tab, which is a PostgreSQL query to find the top 5 referees by the percentage of games lost by the visiting team. The query is as follows:

```
-- 3. Top 5 de árbitros en cuyos juegos pierde el equipo visitante.
SELECT o.official_id,
       CONCAT_WS(' ', o.first_name, o.last_name) AS arbitro,
       COUNT(*) FILTER (WHERE g.away_result = 'L') AS derrotas_visitante,
       COUNT(*) AS partidos_arbitrados,
       ROUND(100.0 * COUNT(*) FILTER (WHERE g.away_result = 'L') / COUNT(*), 2) AS pct_derrota_visitante
FROM game_official go
JOIN official o ON o.official_id = go.official_id
JOIN game g ON g.game_id = go.game_id
GROUP BY o.official_id, o.first_name, o.last_name
ORDER BY derrotas_visitante DESC
LIMIT 5;
```

The 'Data Output' tab shows the results of the query, displaying 5 rows. The columns are: official_id [PK] bigint, arbitro text, derrotas_visitante bigint, partidos_arbitrados bigint, and pct_derrota_visitante numeric. The status bar at the bottom indicates 'Successfully run. Total query runtime: 216 msec. 5 rows affected.'

official_id [PK] bigint	arbitro text	derrotas_visitante bigint	partidos_arbitrados bigint	pct_derrota_visitante numeric
1199	Tom Washing...	693	1135	61.06
1148	James Capers	685	1082	63.31
1162	Scott Foster	677	1147	59.02
2530	Ed Malloy	674	1117	60.34
2003	Pat Fraher	663	1076	61.62

4. ¿Qué equipo maneja los salarios más altos actualmente (última temporada)? Compare esa información ahora con los equipos con los jugadores más valiosos, ¿existe alguna relación? ¿Los equipos con salarios más altos tienen a los jugadores más valiosos?

Equipo	Nómina	Rank gasto	PIE estrella	Rank talento	Brecha
GSW	\$170.5M	1	0.1760	7	-6
BKN	\$168.9M	2	0.1700	9	-7
PHI	\$147.4M	3	0.2110	1	+2
LAC	\$138.6M	4	0.1690	10	-6
LAL	\$138.3M	5	0.1940	3	+2

Los equipos con salarios más altos no tienen necesariamente a los jugadores más valiosos. Golden State y Brooklyn ocupan los dos primeros puestos en gasto y solo el séptimo y noveno en talento. Philadelphia, con la tercera nómina, tiene al jugador con mayor PIE de la liga.

```

-- 4a. Equipo con la nómina mas alta en la ultima temporada, comparado con los equipos que tienen los jugadores mas valiosos.
WITH nomina AS (
  SELECT ts.team_id,
         t.full_name,
         ts.total_salary,
         Rank() Over (ORDER BY ts.total_salary DESC) AS rank_nomina
  FROM team_salary ts
  JOIN team t ON t.team_id = ts.team_id
  WHERE ts.season_id = '2020-21'
),
valor AS (
  SELECT ps.team_id,
         MAX(p.salary_value) AS salario_mas_alto,
         MAX(p.pie) AS pie_estrella
  FROM player_salary ps
  JOIN player p ON p.player_id = ps.player_id
  WHERE ps.season_id = '2020-21'
  GROUP BY ps.team_id
)
SELECT nomina.team_id, nomina.full_name, nomina.rank_nomina, valor.salario_mas_alto, valor.pie_estrella, nomina.rank_nomina - valor.pie_estrella AS brecha
FROM nomina
JOIN valor ON nomina.team_id = valor.team_id

```

id	full_name	rank_nomina	salario_mas_alto	pie_estrella	brecha
1	GSW	1	43006342.00	0.1760	-6
2	BKN	2	41254920.00	0.1700	-7
3	PHI	3	3438880.00	0.2110	2
4	LAC	4	34040412.00	0.1690	-6
5	LAL	5	29219964.00	0.1940	-4
6	UTA	6	34021732.00	0.1980	-4
7	MIL	7	33051724.00	0.1960	-4
8	MIA	8	34379108.00	0.1900	-5
9	NOP	9	29592695.00	0.1540	-7

```

-- 4b. Medida objetiva de la relacion entre gasto y talento.
WITH AS (
  SELECT team_id, total_salary FROM team_salary WHERE season_id = '2020-21'
),
PIE AS (
  SELECT ps.team_id, MAX(p.pie) AS pie_estrella
  FROM player_salary ps
  JOIN player p ON p.player_id = ps.player_id
  WHERE ps.season_id = '2020-21'
  GROUP BY ps.team_id
)
SELECT ROUND(CORR(n.total_salary, v.pie_estrella)::numeric, 4) AS correlacion_nomina_valor,
COUNT(*) AS equipos_comparados
FROM AS JOIN PIE ON AS.team_id = v.team_id

```

correlacion_nomina_valor	equipos_comparados
0.4791	30

5. ¿Cuál fue la temporada con más partidos en la historia de la NBA? ¿Cuál fue la temporada que más se ha prolongado en fechas?

Temporada	Partidos	Inicio	Fin	Días
2019-20	1,059	2019-10-22	2020-08-14	297
1978-79	902	1978-10-13	1979-04-08	177
2017-18	1,230	2017-10-17	2018-04-11	176

No existe una respuesta única. Trece temporadas empatan en 1,230 partidos, resultado de la fórmula de 30 equipos por 82 partidos dividido entre dos.

The image contains two screenshots of the pgAdmin 4 interface, showing SQL queries and their results for NBA seasons.

Top Screenshot: The query is titled "5a. Temporada con mas partidos en la historia de la NBA." and is as follows:

```

-- 5a. Temporada con mas partidos en la historia de la NBA.
WITH s AS (
    SELECT season_id,
           COUNT(*) AS partidos,
           MIN(game_date) AS inicio,
           MAX(game_date) AS fin,
           MAX(game_date) - MIN(game_date) AS dias
    FROM game
    GROUP BY season_id
)
SELECT season_id, partidos, inicio, fin, dias
FROM s
WHERE partidos = (SELECT MAX(partidos) FROM s)
ORDER BY season_id;

```

The results show 13 rows, all with 1230 partidos. The seasons listed are 2004-05, 2005-06, 2006-07, 2007-08, 2008-09, 2009-10, 2010-11, 2013-14, and 2014-15.

Bottom Screenshot: The query is titled "5b. Temporada que mas se prolongo en fechas." and is as follows:

```

-- 5b. Temporada que mas se prolongo en fechas.
WITH s AS (
    SELECT season_id,
           COUNT(*) AS partidos,
           MIN(game_date) AS inicio,
           MAX(game_date) AS fin,
           MAX(game_date) - MIN(game_date) AS dias
    FROM game
    GROUP BY season_id
)
SELECT season_id, partidos, inicio, fin, dias
FROM s
ORDER BY dias DESC
LIMIT 5;

```

The results show 5 rows, ordered by the number of days (dias) in descending order. The seasons listed are 2019-20 (297 dias), 1978-79 (177 dias), 2018-19 (176 dias), 2017-18 (176 dias), and 1977-78 (173 dias).

6. ¿Cuál es el equipo que más diferencia de puntos a favor obtuvo en promedio por partido para la temporada 2017? ¿Y para la temporada 2018?

Temporada	Equipo	Margen promedio
2017-18	Houston Rockets	+8.48
2017-18	Toronto Raptors	+7.78
2018-19	Milwaukee Bucks	+8.87
2018-19	Golden State Warriors	+6.46

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'nba_project' database. The main window shows a SQL query being executed in the 'analysis_queries.sql' file. The query is as follows:

```
-- 6. Equipo con mayor diferencia de puntos a favor por partido en las temporadas 2017-18 y 2018-19.
WITH margenes AS (
  SELECT season_id, home_team_id AS team_id, home_points - away_points AS margen
  FROM game
  UNION ALL
  SELECT season_id, away_team_id, away_points - home_points
  FROM game
),
promedios AS (
  SELECT season_id,
         team_id,
         ROUND(AVG(margen), 2) AS margen_promedio,
         COUNT(*) AS partidos,
         RANK() OVER (PARTITION BY season_id ORDER BY AVG(margen) DESC) AS posicion
  FROM margenes
  WHERE season_id IN ('2017-18', '2018-19')
  GROUP BY season_id, team_id
```

The 'Data Output' tab shows the results of the query:

season_id	full_name	margen_promedio	partidos
2017-18	HOU	8.48	82
2017-18	TOR	7.78	82
2017-18	GSW	5.98	82
2018-19	MIL	8.87	82
2018-19	GSW	6.46	82
2018-19	TOR	6.09	82

The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.096'.

7. ¿Quién es el jugador más valioso del draft1 del 2018 hoy en día (última temporada)?

Jugador	Pick	PPG 2020-21	AST	REB
Luka Doncic	3	27.70	8.60	8.00
Trae Young	5	25.30	9.40	3.90
Collin Sexton	8	24.30	4.40	3.10
Shai Gilgeous-Alexander	11	23.70	5.90	4.70

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'nba_project' database. The main pane shows a SQL query and its results. The query is as follows:

```
-- 7. Jugador mas valioso del draft 2018 en la ultima temporada.
SELECT d.player_name,
       d.overall_pick,
       t.full_name AS equipo_que_lo_selecciono,
       p.pie AS pie_carrera,
       s.points_per_game,
       s.assists_per_game,
       s.rebounds_per_game,
       s.season_id AS temporada_api
FROM draft_selection d
JOIN player p ON p.player_id = d.player_id
LEFT JOIN team t ON t.team_id = d.team_id
LEFT JOIN player_season_stats s ON s.player_id = d.player_id
```

The results table shows 10 rows of data, with Luka Doncic at the top:

player_name	overall_pick	equipo_que_lo_selecciono	pie_carrera	points_per_game	assists_per_game	rebounds_per_game	temporada_api
Luka Doncic	3	ATL	0.1870	27.70	8.60	8.00	2020-21
Trae Young	5	DAL	0.1510	25.30	9.40	3.90	2020-21
Collin Sexton	8	CLE	0.1120	24.30	4.40	3.10	2020-21
Shai Gilgeous-Alexander	11	CHA	0.1460	23.70	5.90	4.70	2020-21
Michael Porter Jr.	14	DEN	0.1160	19.00	1.10	7.30	2020-21
Khyri Thomas	38	PHI	[null]	16.40	5.00	3.60	2020-21
Gary Trent Jr.	37	SAC	0.0690	15.30	1.40	2.60	2020-21
Devonte' Graham	34	ATL	0.0900	14.80	5.40	2.70	2020-21
Deandre Ayton	1	PHX	0.1290	14.40	1.40	10.50	2020-21
Jaren Jackson Jr.	4	MEM	0.0890				

The status bar at the bottom indicates: 'Total rows: 10', 'Query complete 00:00:00.083', and 'Successfully run. Total query runtime: 83 msec. 10 rows affected.'

Luka Doncic es el jugador más valioso del draft de 2018 en la última temporada, con 27.70 puntos por partido y el PIE más alto de su clase.

8. Calcule el top 5 de los estados que más salarios pagaron durante las temporadas 2020/2021 y 2021/2022.

Estado	Equipos	Salarios totales	Promedio por equipo
California	4	\$1,077.8M	\$269.4M
Texas	3	\$624.2M	\$208.1M
New York	2	\$480.5M	\$240.2M
Florida	2	\$435.3M	\$217.7M

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including the 'nba_project' database. The main window shows a SQL query in the 'Query' tab, which is a PostgreSQL query to find the top 5 states by total salary paid during the 2020-21 and 2021-22 seasons. The query uses a CTE named 'equipo_estado' to join the 'team' and 'salary' tables, filtering for the specified seasons. The results are displayed in the 'Data Output' tab, showing a table with 5 rows and 4 columns: 'estado', 'equipos', 'salarios_totales', and 'promedio_por_equipo'.

```

-- 8. Top 5 de estados que mas salarios pagaron en 2020-21 y 2021-22.
WITH equipo_estado AS (
  SELECT team_id,
         full_name,
         CASE
           WHEN full_name = 'Atlanta Hawks' THEN 'Georgia'
           WHEN full_name = 'Toronto Raptors' THEN 'Ontario (Canada)'
           ELSE state
         END AS estado
  FROM team
  WHERE is_current

```

estado	equipos	salarios_totales	promedio_por_equipo
California	4	1077788518.00	269447129.50
Texas	3	624189677.00	208063225.67
New York	2	480467875.00	240233937.50
Florida	2	435331546.00	217665773.00
Wisconsin	1	278821035.00	278821035.00

Total rows: 5 Query complete 00:00:00.073 CRLF Ln 200, Col 1

VI. Etapa 3: Preguntas Propias

Las trece consultas se encuentran en database/fase_3_queries.sql, organizadas en las tres dimensiones definidas en la sección 1.

Rendimiento sostenido

1. Consistencia con piso alto

La primera versión de esta consulta ordenaba únicamente por desviación estándar del porcentaje de victorias. El resultado colocaba a Sacramento en el segundo lugar con 39.8% de victorias promedio: un equipo consistentemente malo. La consistencia por sí sola no es una virtud; solo importa acompañada de rendimiento.

Se exigió por tanto un piso mínimo de 40% de victorias en la peor de las seis temporadas. Solo nueve de treinta equipos superan el filtro:

Equipo	% victorias	Piso	Variabilidad	Margen
BOS	0.6111	0.5000	0.0649	+3.60
LAC	0.6096	0.5122	0.0576	+3.60
SAS	0.6024	0.4444	0.1518	+3.23
MIL	0.5982	0.4024	0.1397	+3.36
UTA	0.5948	0.4878	0.0568	+4.45
DEN	0.5585	0.4024	0.0972	+1.71

Utah tiene el piso más alto de la liga con 48.8% de victorias en su peor temporada, la variabilidad más baja del grupo y el mejor margen de puntos. Boston y los Clippers tienen mejor promedio, pero cayeron más bajo en sus peores años.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

Servers (1)

- PostgreSQL 18
 - Databases (3)
 - lab04
 - Cast
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - nba_project
 - Cast
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions
 - postgres
 - Cast
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Subscriptions

Query Editor

nba_project/postgres@PostgreSQL 18

Query

```

62 */
63 SELECT t.abbreviation AS equipo,
64        t.full_name,
65        ROUND(AVG(a.pct), 4) AS pct_promedio,
66        ROUND(MIN(a.pct), 4) AS piso_peor_temporada,
67        ROUND(STDDEV_SAMP(a.pct), 4) AS variabilidad,
68        ROUND(AVG(a.margen), 2) AS margen_promedio
69 FROM analysis_team_season a
70 JOIN team t ON t.team_id = a.team_id
71 WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
72 GROUP BY t.team_id, t.abbreviation, t.full_name
73 HAVING COUNT(*) = 6 AND MIN(a.pct) >= 0.468
74 ORDER BY pct_promedio DESC

```

Data Output

Showing rows: 1 to 9

Page No: 1 of 1

	equipo character varying (10)	full_name character varying (100)	pct_promedio numeric	piso_peor_temporada numeric	variabilidad numeric	margen_promedio numeric
1	BOS	BOS	0.6111	0.5000	0.0649	3.60
2	LAC	LAC	0.6096	0.5122	0.0576	3.60
3	SAS	SAS	0.6024	0.4444	0.1518	3.23
4	MIL	MIL	0.5982	0.4024	0.1397	3.36
5	UTA	UTA	0.5948	0.4878	0.0568	4.45
6	DEN	DEN	0.5585	0.4024	0.0972	1.71
7	IND	IND	0.5511	0.4583	0.0579	1.34
8	POR	POR	0.5469	0.4730	0.0642	1.27
9	MIA	MIA	0.5426	0.4756	0.0489	0.99

Total rows: 9 Query complete 00:00:00.119

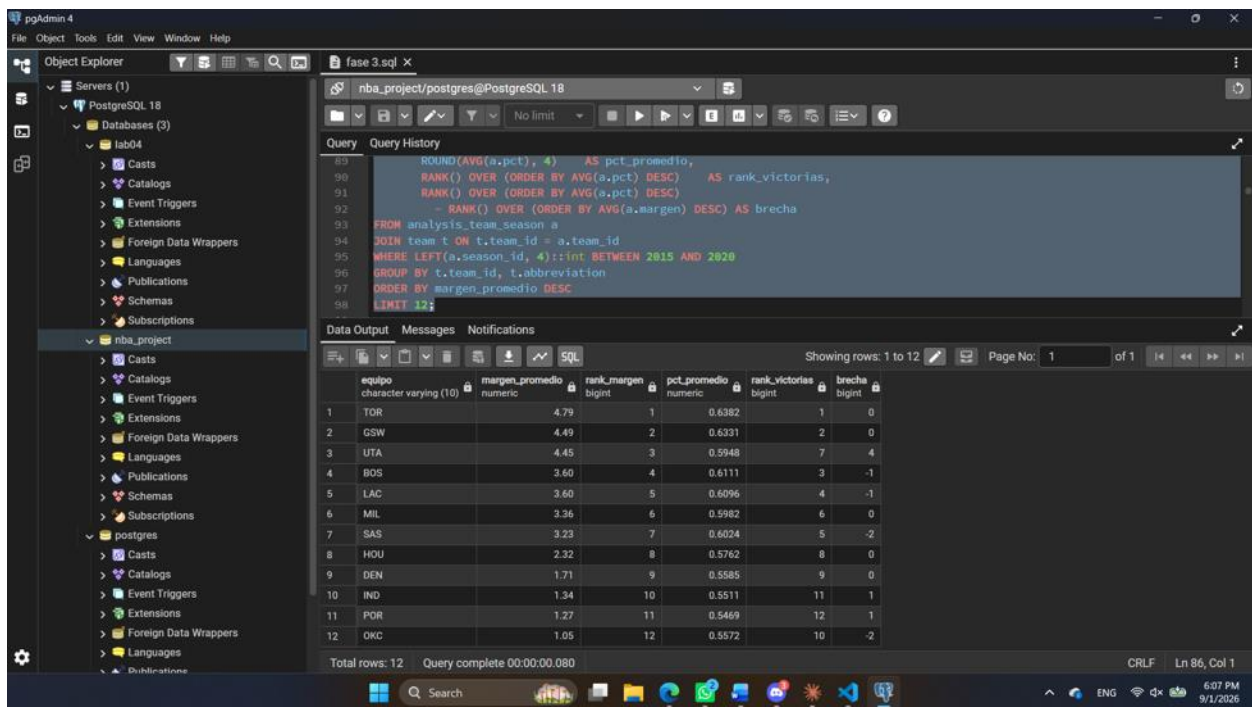
CRLF Ln 63, Col 1

6:05 PM 9/1/2026

2. Diferencial de puntos frente al récord

El porcentaje de victorias es ruidoso: un equipo puede acumular triunfos ajustados por suerte. El diferencial de puntos es una medida más estable del nivel real, por lo que comparamos la posición en ambos rankings.

Utah aparece tercero en margen con +4.45 pero séptimo en victorias, una brecha de cuatro posiciones a su favor: su récord subestima su nivel real. San Antonio y Oklahoma City muestran el patrón contrario.



The screenshot shows the pgAdmin 4 interface with a SQL query executed against the 'nba_project/postgres@PostgreSQL 18' database. The query calculates the difference in ranking between points percentage and wins percentage for each team. The results are displayed in a table with 12 rows.

```
Query
```

```
80      ROUND(AVG(a.pct), 4) AS pct_promedio,
81      RANK() OVER (ORDER BY AVG(a.pct) DESC) AS rank_victorias,
82      RANK() OVER (ORDER BY AVG(a.pct) DESC)
83      - RANK() OVER (ORDER BY AVG(a.margen) DESC) AS brecha
84 FROM analysis_team_season a
85 JOIN team t ON t.team_id = a.team_id
86 WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
87 GROUP BY t.team_id, t.abbreviation
88 ORDER BY margen_promedio DESC
89 LIMIT 12;
```

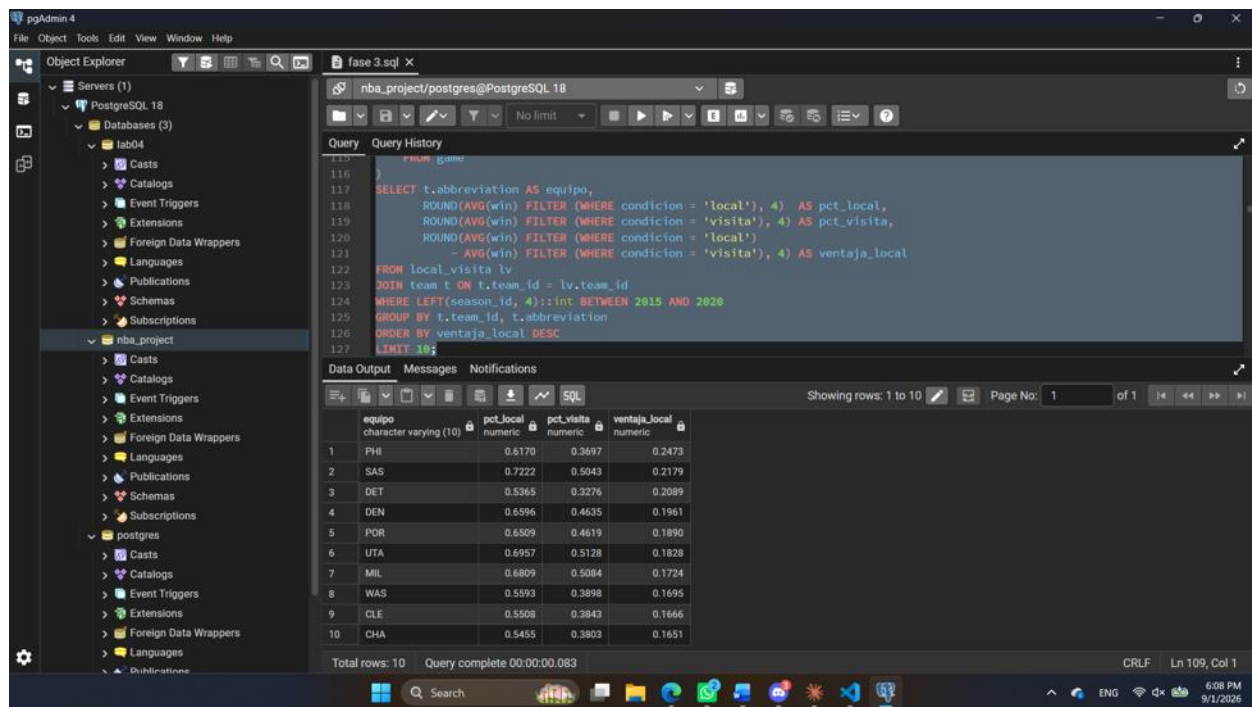
	equipo character varying (10)	margen_promedio numeric	rank_margen bigint	pct_promedio numeric	rank_victorias bigint	brecha bigint
1	TOR	4.79	1	0.6382	1	0
2	GSW	4.49	2	0.6331	2	0
3	UTA	4.45	3	0.5948	7	4
4	BOS	3.60	4	0.6111	3	-1
5	LAC	3.60	5	0.6096	4	-1
6	MIL	3.36	6	0.5982	6	0
7	SAS	3.23	7	0.6024	5	-2
8	HOU	2.32	8	0.5762	8	0
9	DEN	1.71	9	0.5585	9	0
10	IND	1.34	10	0.5511	11	1
11	POR	1.27	11	0.5469	12	1
12	OKC	1.05	12	0.5572	10	-2

Total rows: 12 Query complete 00:00:00.080

3. Ventaja de local

Esta consulta introduce la dimensión comercial, que conecta directamente con una decisión de inversión: un equipo que gana notoriamente más en casa sostiene mejor la asistencia, la venta de boletos y el valor de los patrocinios locales.

Philadelphia lidera con 24.7 puntos porcentuales de diferencia. San Antonio destaca en magnitud absoluta, ganando el 72.2% de sus partidos en casa. Utah aparece sexta, pero con el detalle relevante de que también gana más de la mitad de sus partidos como visitante.



The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with a tree view of the database structure, including 'Servers (1)', 'PostgreSQL 18', 'Databases (3)', and 'nba_project'. The main pane shows a SQL query in the 'Query' tab, which is a complex query involving a join between 'local_visita' and 'team' tables, filtering by season_id, and calculating local and visitor win percentages. The 'Data Output' pane shows the results of the query, displaying 10 rows of data with columns: equipo, pct_local, pct_visita, and ventaja_local.

```
115 --
116 --
117 SELECT t.abbreviation AS equipo,
118        ROUND(AVG(win) FILTER (WHERE condition = 'local'), 4) AS pct_local,
119        ROUND(AVG(win) FILTER (WHERE condition = 'visita'), 4) AS pct_visita,
120        ROUND(AVG(win) FILTER (WHERE condition = 'local')
121              - AVG(win) FILTER (WHERE condition = 'visita'), 4) AS ventaja_local
122 FROM local_visita lv
123 JOIN team t ON t.team_id = lv.team_id
124 WHERE LEFT(season_id, 4)::int BETWEEN 2015 AND 2020
125 GROUP BY t.team_id, t.abbreviation
126 ORDER BY ventaja_local DESC
127 LIMIT 10;
```

equipo	pct_local	pct_visita	ventaja_local
PHI	0.6170	0.3697	0.2473
SAS	0.7222	0.5043	0.2179
DET	0.5365	0.3276	0.2089
DEN	0.6596	0.4635	0.1961
POR	0.6509	0.4619	0.1890
UTA	0.6957	0.5128	0.1828
MIL	0.6809	0.5084	0.1724
WAS	0.5593	0.3898	0.1695
CLE	0.5508	0.3843	0.1666
CHA	0.5455	0.3803	0.1651

Total rows: 10 Query complete 00:00:00.083

Valor por dinero

1. Victorias por millón de dólares

Dos equipos con 47 victorias no valen lo mismo si uno las consiguió con 30 millones menos de nómina.

Equipo	Victorias	Nómina	Victorias por millón
NYK	41	\$97.6M	0.420
PHX	47	\$128.9M	0.365
ATL	41	\$118.6M	0.346
UTA	47	\$136.0M	0.345
DEN	44	\$129.2M	0.341
MIL	46	\$135.3M	0.340

Nueva York encabeza con 41 victorias: es eficiente porque gasta poco, no porque gane mucho.

Phoenix y Utah aparecen segundo y cuarto con 47 victorias cada uno, el mejor registro del grupo.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure for 'nba_project'. The main pane displays a SQL query and its results. The query is as follows:

```
SELECT t.abbreviation AS equipo,
       t.full_name,
       a.victorias,
       ROUND(ts.total_salary / 1000000, 1) AS nomina_millones,
       ROUND(a.victorias / NULLIF(ts.total_salary / 1000000, 0), 3)
       AS victorias_por_millon
FROM analysis.team_season a
JOIN team_salary ts ON ts.team_id = a.team_id AND ts.season_id = a.season_id
JOIN team t ON t.team_id = a.team_id
WHERE a.season_id = '2020-21'
ORDER BY victorias_por_millon DESC;
```

The results are displayed in a table with the following columns: equipo, full_name, victorias, nomina_millones, and victorias_por_millon. The table shows 11 rows of data, with the top 6 rows corresponding to the teams listed in the table above.

equipo	full_name	victorias	nomina_millones	victorias_por_millon
NYK	NYK	41	97.6	0.420
PHX	PHX	47	128.9	0.365
ATL	ATL	41	118.6	0.346
UTA	UTA	47	136.0	0.345
DEN	DEN	44	129.2	0.341
MIL	MIL	46	135.3	0.340
PHI	PHI	49	147.4	0.332
LAC	LAC	44	138.6	0.317
DAL	DAL	40	127.5	0.314
CHA	CHA	33	108.2	0.305
MIA	MIA	40	134.3	0.298

The bottom of the screenshot shows the Windows taskbar with the date and time as 6:08 PM on 9/1/2026.

2. Talento por encima de lo que la nómina sugiere

Si un equipo tiene mejores jugadores de los que su gasto indicaría, obtuvo talento por debajo del precio de mercado. Ese es el perfil clásico de una buena inversión.

Equipo	Rank gasto	PIE estrella	Rank talento	Brecha a favor
DAL	23	0.1870	6	+17
NYK	29	0.1640	13	+16
DEN	17	0.2050	2	+15
ATL	25	0.1610	14	+11
MIL	7	0.1960	3	+4
PHI	3	0.2110	1	+2

Dallas presenta la mayor brecha de la liga, efecto directo de tener a Luka Doncic con contrato de novato. Denver le sigue con quince posiciones. Este resultado matiza la conclusión de la pregunta 4: la correlación entre nómina y talento es positiva pero moderada, y las excepciones son precisamente donde está la oportunidad.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure for 'PostgreSQL 18'. The central pane shows a SQL query being executed. The query uses window functions to rank teams by total salary and then calculates the difference between their talent rank and salary rank. The bottom pane displays the results of the query as a table with 12 rows.

equipo	nomina_millones	rank_gasto	pie_estrella	rank_talento	brecha_a_favor	
1	DAL	127.5	23	0.1870	6	17
2	NYK	97.6	29	0.1640	13	16
3	DEN	129.2	17	0.2050	2	15
4	ATL	118.6	25	0.1610	14	11
5	CHI	129.0	18	0.1720	8	10
6	OKC	91.6	30	0.1460	21	9
7	MIL	135.3	7	0.1960	3	4
8	MIA	134.3	8	0.1900	5	3
9	PHI	147.4	3	0.2110	1	2
10	CLE	128.6	20	0.1530	18	2
11	POR	131.4	13	0.1650	12	1
12	LAL	138.3	5	0.1940	4	1

Total rows: 12 Query complete 00:00:00.079 CRLF Ln 167, Col 1

3. Margen financiero hacia adelante

Un equipo bueno pero con la nómina comprometida al máximo tiene poca capacidad de reforzarse. San Antonio y Nueva York tienen menos de la mitad de su nómina comprometida para 2021-22, es decir, máxima flexibilidad.

Utah y Milwaukee no aparecen entre los doce con más margen: ambos tienen su núcleo bajo contrato a largo plazo. Presentamos este resultado como argumento en contra de nuestra recomendación. La lectura alternativa es que un núcleo asegurado también significa continuidad y menor riesgo de perder jugadores clave.

The screenshot shows the pgAdmin 4 interface with a SQL query executed against the 'nba_project' database. The query calculates the percentage of salary committed for the 2021-22 season relative to the 2020-21 salary for various NBA teams. The results are displayed in a table with columns: equipo, nomina_2020_21, comprometido_2021_22, and pct_comprometido.

equipo	nomina_2020_21	comprometido_2021_22	pct_comprometido
1 SAS	127.9	58.2	45.5
2 NYK	97.6	48.0	49.2
3 OKC	91.6	49.8	54.4
4 MIA	134.3	77.8	58.0
5 MEM	131.6	87.0	66.2
6 DAL	127.5	85.0	66.6
7 NOP	133.7	91.4	68.4
8 TOR	128.5	88.8	69.1
9 HOU	132.8	92.8	69.9
10 CLE	128.6	90.7	70.5
11 CHI	129.0	96.3	74.7
12 CHA	108.2	83.1	76.8

Crecimiento y talento joven

1. Trayectoria ascendente

Comparar únicamente la primera contra la última temporada confunde un repunte puntual con una mejora sostenida. Se utilizó la pendiente de una regresión lineal sobre las seis temporadas mediante REGR_SLOPE, que considera todos los puntos y no solo los extremos.

Philadelphia tiene la pendiente más pronunciada (0.1007), seguida de los Lakers y Brooklyn. Utah tiene pendiente positiva pero modesta (0.0233), porque partía de un nivel ya alto: es más difícil mejorar desde 59% de victorias que desde 30%.

pgAdmin 4

Object Explorer

Servers (1)

PostgreSQL 18

Databases (3)

lab04

Castes

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

nba_project

Castes

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas

Subscriptions

postgres

Castes

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Subscriptions

Query

Query History

```
234 SELECT t.abbreviation AS equipo,
235        t.full_name,
236        ROUND(REGR_SLOPE(a.pct, LEFT(a.season_id, 4)::int)::numeric, 4)
237        AS pendiente_anual,
238        ROUND(AVG(a.pct), 4) AS pct_promedio,
239        ROUND(MAX(a.pct) FILTER (WHERE a.season_id = '2020-21'), 4) AS pct_2020_21
240 FROM analysis_team_season a
241 JOIN team t ON t.team_id = a.team_id
242 WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
243 GROUP BY t.team_id, t.abbreviation, t.full_name
```

Data Output

Messages

Notifications

	equipo	full_name	pendiente_anual	pct_promedio	pct_2020_21
	character varying (10)	character varying (100)	numeric	numeric	numeric
1	PHI	PHI	0.1007	0.4982	0.6806
2	LAL	LAL	0.0841	0.4461	0.5417
3	BKN	BKN	0.0823	0.4154	0.6528
4	PHX	PHX	0.0673	0.3633	0.6528
5	MIL	MIL	0.0612	0.5982	0.6389
6	DEN	DEN	0.0448	0.5585	0.6111
7	DAL	DAL	0.0240	0.4564	0.5556
8	UTA	UTA	0.0233	0.5948	0.6528
9	NYK	NYK	0.0163	0.3695	0.5694
10	NOP	NOP	0.0042	0.4359	0.4306
11	MIA	MIA	0.0028	0.5426	0.5556
12	LAC	LAC	0.0021	0.6096	0.6111

Total rows: 12 Query complete 00:00:00.080

Showing rows: 1 to 12 Page No: 1 of 1

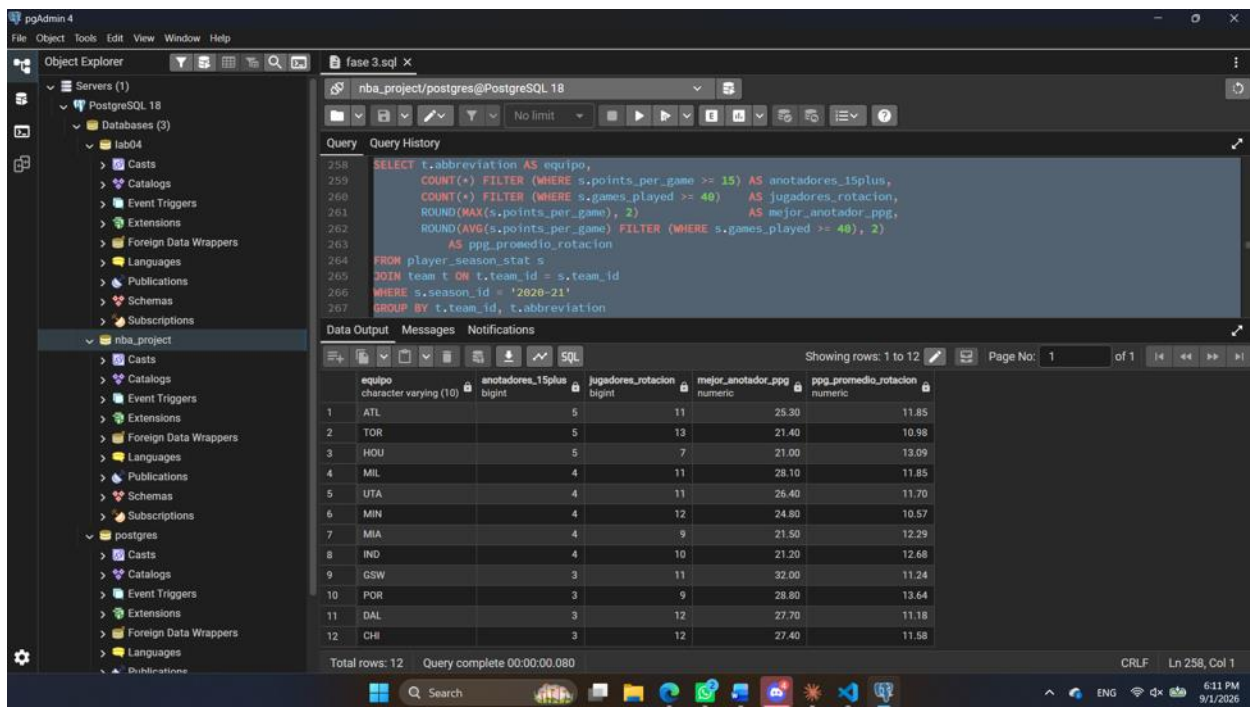
CR/LF Ln 234, Col 1

6:10 PM 9/1/2026

2. Calidad del roster actual

Las consultas anteriores miran al pasado. Esta evalúa el plantel con el que el equipo efectivamente jugó 2020-21, usando estadísticas reales obtenidas del API en lugar de promedios de carrera.

Atlanta, Toronto y Houston tienen cinco jugadores con 15 o más puntos por partido. Utah tiene cuatro, pero con once jugadores de rotación con 40 o más partidos disputados, lo que indica profundidad y no dependencia de un quinteto corto.



The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with the 'nba_project' database selected. The central pane shows a SQL query in the 'Query' tab, and the bottom pane displays the 'Data Output' table.

Query:

```
SELECT t.abbreviation AS equipo,
COUNT(*) FILTER (WHERE s.points_per_game >= 15) AS anotadores_15plus,
COUNT(*) FILTER (WHERE s.games_played >= 40) AS jugadores_rotacion,
ROUND(MAX(s.points_per_game), 2) AS mejor_anotador_ppg,
ROUND(AVG(s.points_per_game) FILTER (WHERE s.games_played >= 40), 2)
AS ppg_promedio_rotacion
FROM player_season_stat s
JOIN team t ON t.team_id = s.team_id
WHERE s.season_id = '2020-21'
GROUP BY t.team_id, t.abbreviation
```

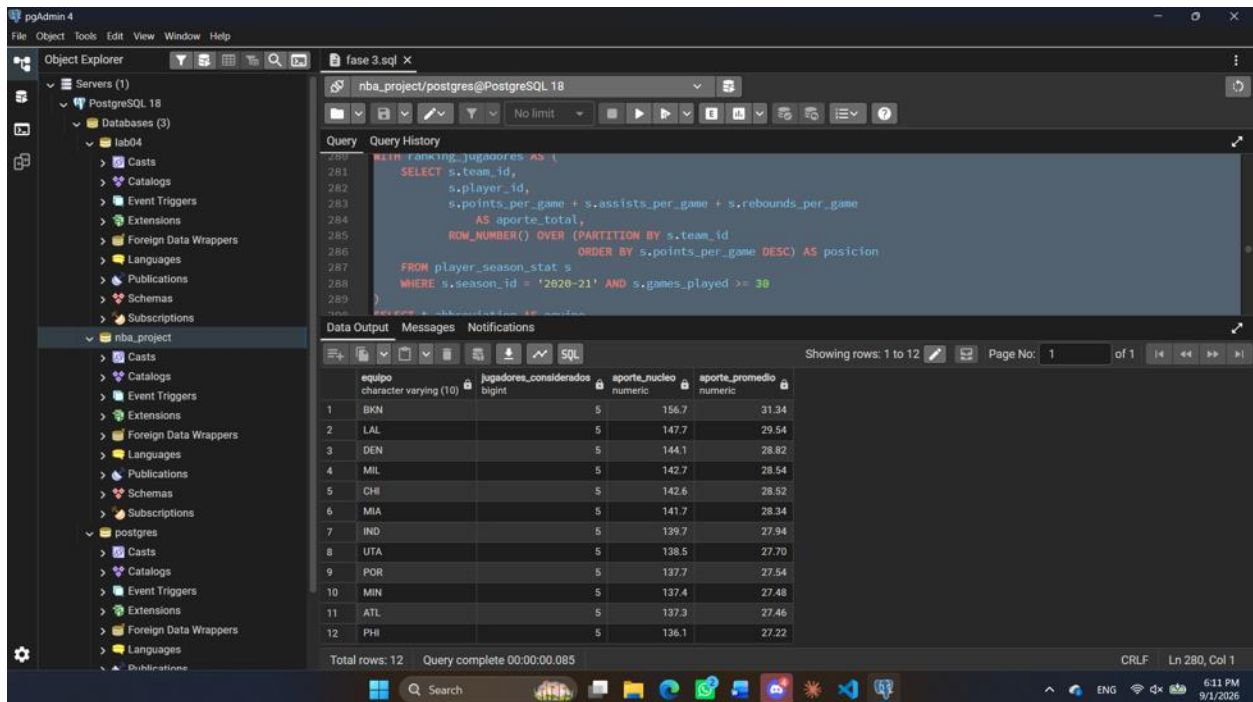
Data Output Table:

	equipo character varying (10)	anotadores_15plus bigint	jugadores_rotacion bigint	mejor_anotador_ppg numeric	ppg_promedio_rotacion numeric
1	ATL	5	11	25.30	11.85
2	TOR	5	13	21.40	10.98
3	HOU	5	7	21.00	13.09
4	MIL	4	11	28.10	11.85
5	UTA	4	11	26.40	11.70
6	MIN	4	12	24.80	10.57
7	MIA	4	9	21.50	12.29
8	IND	4	10	21.20	12.68
9	GSW	3	11	32.00	11.24
10	POR	3	9	28.80	13.64
11	DAL	3	12	27.70	11.18
12	CHI	3	12	27.40	11.58

Total rows: 12 Query complete 00:00:00.080 CRLF Ln 258, Col 1

3. Producción combinada del núcleo

Un núcleo fuerte y balanceado sostiene el rendimiento mejor que una estrella rodeada de suplentes. Brooklyn lidera con amplio margen, resultado de haber concentrado tres estrellas. Utah aparece octava, lo que confirma que su fortaleza no está en la producción bruta sino en la distribución del esfuerzo.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including 'Databases (3)' and 'nba_project'. The main window shows a SQL query in the 'Query' tab, which is a window function query ranking players by team. The 'Data Output' tab shows the results of the query, displaying 12 rows of data. The query is as follows:

```
WITH ranking_jugadores AS (
  SELECT s.team_id,
         s.player_id,
         s.points_per_game + s.assists_per_game + s.rebounds_per_game
         AS aporte_total,
         ROW_NUMBER() OVER (PARTITION BY s.team_id
                             ORDER BY s.points_per_game DESC) AS posicion
  FROM player_season_stat s
  WHERE s.season_id = '2020-21' AND s.games_played >= 30
)
```

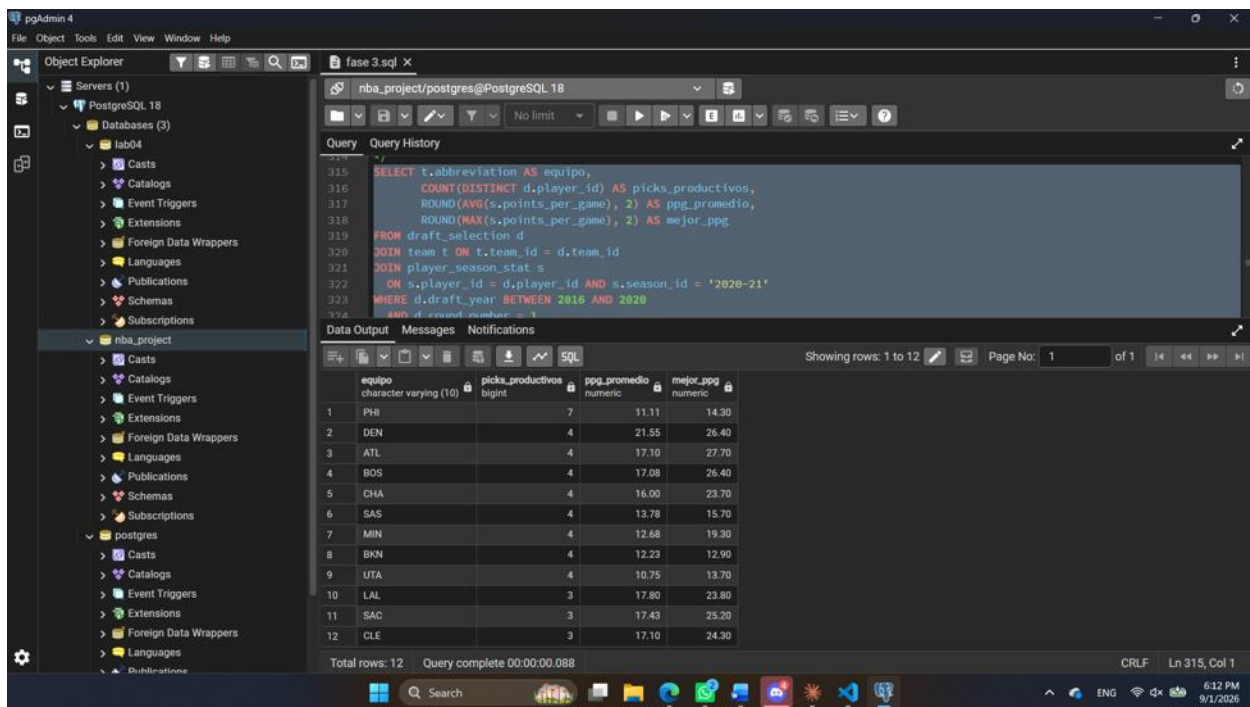
The results table has the following columns: equipo, jugadores_considerados, aporte_nucleo, and aporte_promedio. The data is as follows:

equipo	jugadores_considerados	aporte_nucleo	aporte_promedio
1 BKN	5	156.7	31.34
2 LAL	5	147.7	29.54
3 DEN	5	144.1	28.82
4 MIL	5	142.7	28.54
5 CHI	5	142.6	28.52
6 MIA	5	141.7	28.34
7 IND	5	139.7	27.94
8 UTA	5	138.5	27.70
9 POR	5	137.7	27.54
10 MIN	5	137.4	27.48
11 ATL	5	137.3	27.46
12 PHI	5	136.1	27.22

4. Construcción mediante el draft

El talento formado en el draft cuesta considerablemente menos que el fichado en agencia libre, porque los contratos de novato están limitados por convenio. Un buen historial de draft es una ventaja financiera sostenida.

Philadelphia tiene siete selecciones de primera ronda entre 2016 y 2020 que ya producen, muy por encima del resto. Denver y Atlanta tienen cuatro, pero con promedios de anotación considerablemente más altos: cantidad frente a calidad.



The screenshot shows the pgAdmin 4 interface with a PostgreSQL database named 'nba_project'. A query is executed in the 'Query' tab, and the results are displayed in the 'Data Output' tab. The query calculates statistics for teams based on draft selections between 2016 and 2020.

```
SELECT t.abbreviation AS equipo,
       COUNT(DISTINCT d.player_id) AS picks_productivos,
       ROUND(AVG(s.points_per_game), 2) AS ppg_promedio,
       ROUND(MAX(s.points_per_game), 2) AS mejor_ppg
FROM draft_selection d
JOIN team t ON t.team_id = d.team_id
JOIN player_season_stats s
  ON s.player_id = d.player_id AND s.season_id = '2020-21'
WHERE d.draft_year BETWEEN 2016 AND 2020
AND d.draft_round = 1
```

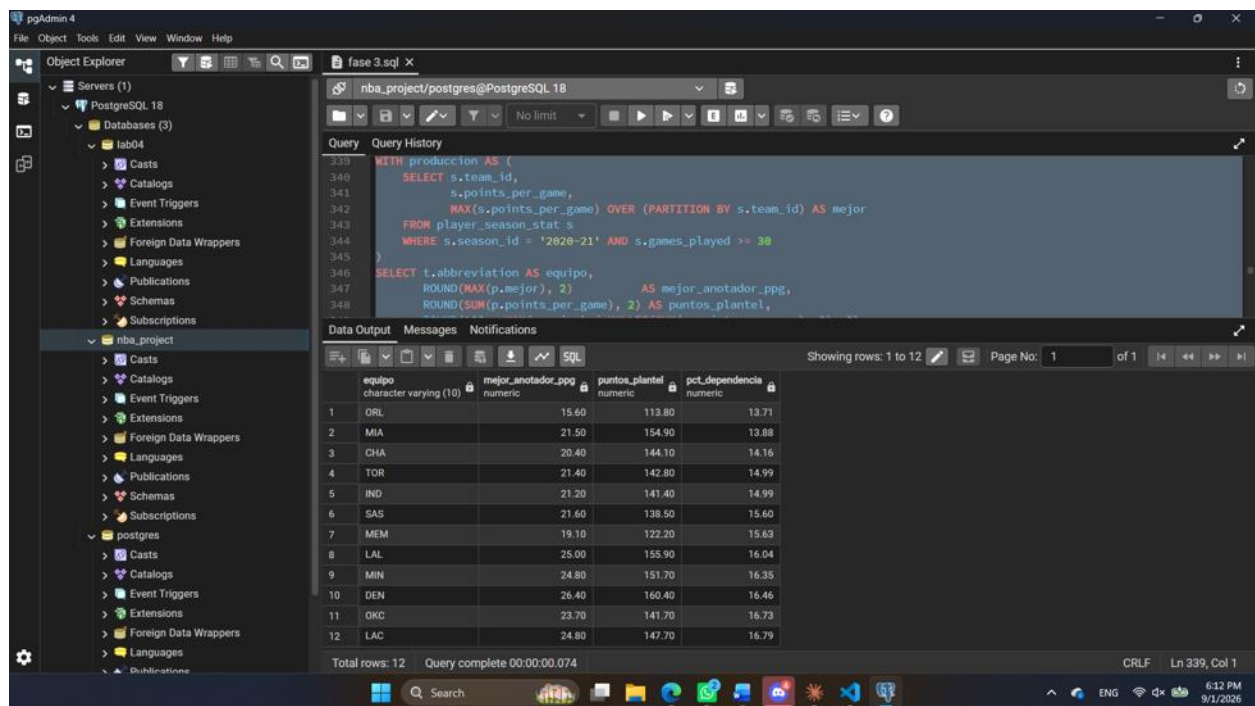
	equipo	picks_productivos	ppg_promedio	mejor_ppg
1	PHI	7	11.11	14.30
2	DEN	4	21.55	26.40
3	ATL	4	17.10	27.70
4	BOS	4	17.08	26.40
5	CHA	4	16.00	23.70
6	SAS	4	13.78	15.70
7	MIN	4	12.68	19.30
8	BKN	4	12.23	12.90
9	UTA	4	10.75	13.70
10	LAL	3	17.80	23.80
11	SAC	3	17.43	25.20
12	CLE	3	17.10	24.30

Total rows: 12 Query complete 00:00:00.088

5. Dependencia de una sola figura

Un equipo cuya producción se concentra en un jugador es frágil: una lesión hunde la temporada completa. Esta consulta funciona como control de riesgo sobre las anteriores.

Orlando, Miami y Charlotte son los menos dependientes, aunque en varios casos porque simplemente no tienen una figura dominante. Utah no aparece entre los doce menos dependientes, lo que indica una concentración algo mayor de lo ideal.



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'nba_project' database. The central pane shows a SQL query in the 'Query History' tab. The query is a complex SQL statement using a CTE named 'production' to calculate the maximum points per game for each team and then calculate the percentage of total team points contributed by that player.

```
WITH production AS (  
    SELECT s.team_id,  
           s.points_per_game,  
           MAX(s.points_per_game) OVER (PARTITION BY s.team_id) AS mejor  
    FROM player_season_stat s  
    WHERE s.season_id = '2020-21' AND s.games_played >= 30  
)  
SELECT t.abbreviation AS equipo,  
       ROUND(MAX(p.mejor), 2) AS mejor_anotador_ppg,  
       ROUND(SUM(p.points_per_game), 2) AS puntos_plantel,  
       ROUND(SUM(p.points_per_game) / MAX(p.mejor), 2) AS pct_dependencia
```

The bottom pane shows the 'Data Output' tab with the results of the query. The results are displayed in a table with 4 columns: 'equipo', 'mejor_anotador_ppg', 'puntos_plantel', and 'pct_dependencia'. The table shows data for 12 teams, ranked by 'pct_dependencia' in ascending order.

equipo	mejor_anotador_ppg	puntos_plantel	pct_dependencia
ORL	15.60	113.80	13.71
MIA	21.50	154.90	13.88
CHA	20.40	144.10	14.16
TOR	21.40	142.80	14.99
IND	21.20	141.40	14.99
SAS	21.60	138.50	15.60
MEM	19.10	122.20	15.63
LAL	25.00	155.90	16.04
MIN	24.80	151.70	16.35
DEN	26.40	160.40	16.46
OKC	23.70	141.70	16.73
LAC	24.80	147.70	16.79

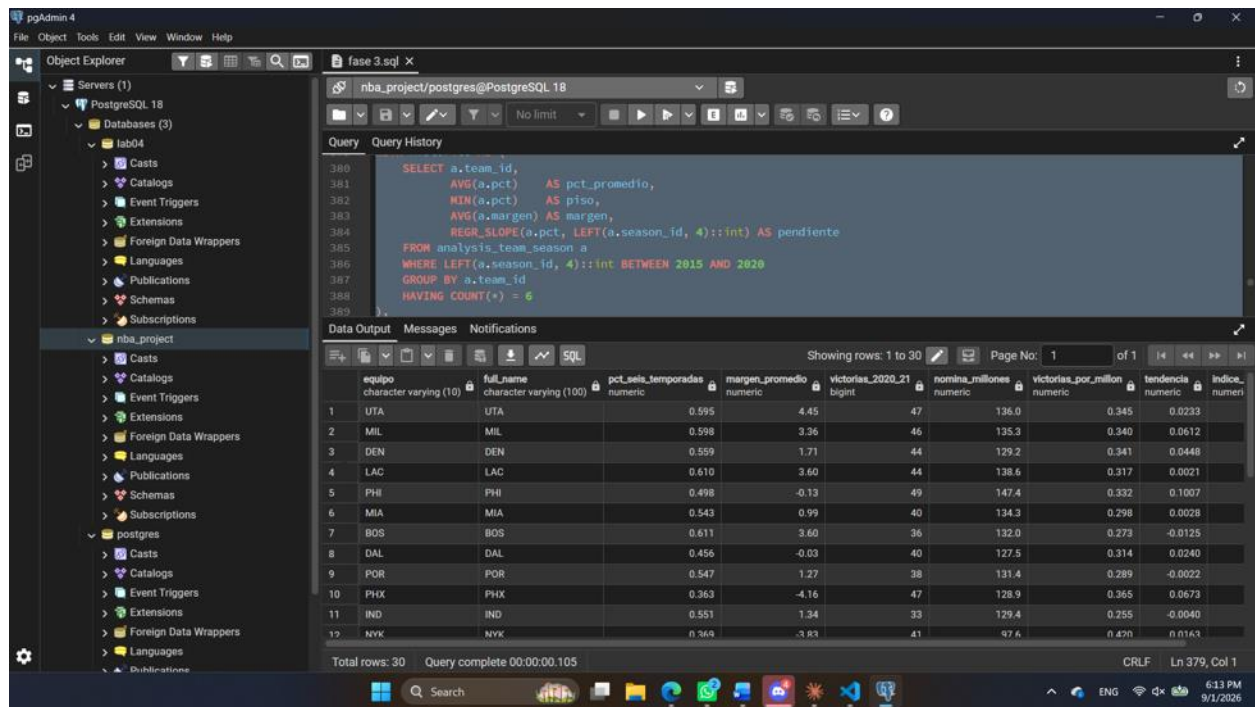
VII. Recomendación y conclusión

Se utilizó el "PERCENT_RANK" para convertir cada valor en una posición relativa entre 0 y 1 dentro de la liga.

Dimensión	Peso	Justificación
Rendimiento	40%	Es la base: sin ganar no hay negocio. Se reparte en 50% récord, 30% margen, 20% piso.
Valor	35%	Distingue al equipo bien gestionado del que solo compra resultados.
Crecimiento	25%	Se invierte para el futuro, pero pesa menos por ser la dimensión más incierta.

Resultado:

Pos.	Equipo	% victorias	Margen	Vict. 2020-21	Nómina	Índice
1	Utah Jazz	0.595	+4.45	47	\$136.0M	0.8483
2	Milwaukee Bucks	0.598	+3.36	46	\$135.3M	0.8279
3	Denver Nuggets	0.559	+1.71	44	\$129.2M	0.7983
4	Los Angeles Clippers	0.610	+3.60	47	\$138.6M	0.7834
5	Philadelphia 76ers	0.498	-0.13	49	\$147.4M	0.7000



El **análisis de sensibilidad** habla que, ya que el índice que creamos busca ciertos resultados, se vieron esquemas alternativos que priorizan cada característica.

Equipo	Mixto	Prioriza rendimiento	Prioriza valor	Prioriza crecimiento
UTA	1	1	1	4
MIL	2	3	2	2
DEN	3	5	3	3
LAC	4	2	6	9
PHI	5	12	5	1

Utah queda primero con el esquema mixto, primero priorizando rendimiento y primero priorizando valor. Solo desciende al cuarto lugar cuando el crecimiento pesa 60%, donde Philadelphia toma la delantera. La conclusión es robusta: no depende de los pesos elegidos.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' shows the database structure. The main pane displays a SQL query in the 'Query' tab. The query is as follows:

```

WITH historico AS (
  SELECT a.team_id, AVG(a.pct) AS pct_promedio, MIN(a.pct) AS piso,
         AVG(a.margen) AS margen,
         REGR_SLOPE(a.pct, LEFT(a.season_id, 4)::int) AS pendiente
  FROM analysis_team_season a
  WHERE LEFT(a.season_id, 4)::int BETWEEN 2015 AND 2020
  GROUP BY a.team_id HAVING COUNT(*) = 6
),
actual AS (
  SELECT a.team_id,
         a.victorias / NULLIF(ts.total_salary / 1000000.0, 0) AS eficiencia
  FROM analysis_team_season a
)

```

The 'Data Output' pane shows the results of the query, displaying 10 rows of team data. The columns are: equipo, mido, pos_mido, pos_rendimiento, pos_valor, and pos_crecimiento.

equipo	mido	pos_mido	pos_rendimiento	pos_valor	pos_crecimiento
1 UTA	0.8483	1	1	1	4
2 MIL	0.8279	2	3	2	2
3 DEN	0.7983	3	5	3	3
4 LAC	0.7834	4	2	6	9
5 PHI	0.7000	5	12	5	1
6 MIA	0.6524	6	7	10	10
7 BOS	0.6434	7	4	14	14
8 DAL	0.6338	8	13	9	7
9 POR	0.6183	9	9	11	12
10 PHX	0.5731	10	24	4	6

Recomendamos invertir en Utah Jazz.

El argumento en una línea: es el equipo que mejor combina un rendimiento sostenido con piso alto, eficiencia de gasto y una tendencia positiva, y su posición no depende de cómo se ponderen esas tres cosas.

Lo relevante es cómo gana el índice. Utah no es primero en ninguna consulta individual, pero está en el grupo alto de las tres dimensiones a la vez: tiene el mejor margen de puntos entre los equipos que superan el filtro de piso, la cuarta mejor eficiencia de gasto y una tendencia positiva.

Se conoce que hay algunos riesgos:

- Hay poco margen financiero ya que la nómina está comprometida al largo plazo.
- No está entre los equipos menos dependientes de una figura individual.
- Su crecimiento es el más modesto de los cuatro primeros del índice, porque partía de un nivel ya alto.

La alternativa de mayor riesgo sería Philadelphia. Es primera en crecimiento, primera en construcción vía draft y con el mejor jugador de la liga por PIE, pero con diferencial de puntos negativo en las seis temporadas y una caída al 12.2% de victorias en su peor año.

VIII. Reproducibilidad

1. Crear el entorno virtual e instalar requirements.txt con Python 3.11 a 3.13.
2. Iniciar PostgreSQL, crear la base nba_project y configurar el archivo .env.
3. Colocar el archivo de datos en data/Data.zip.
4. Ejecutar python scripts/load_csv.py --zip data/Data.zip --dry-run para validar.
5. Ejecutar python scripts/load_csv.py --zip data/Data.zip para cargar.
6. Ejecutar python scripts/load_api.py --season 2020-21 para la ingesta del API.
7. Ejecutar database/verification.sql y confirmar 30 equipos con nómina.
8. Ejecutar database/analysis_queries.sql y database/fase_3_queries.sql.